

AKTS Drug Stability Kinetics in Python

<https://gemini.google.com/app/004e593771fe7b40>

User prompt: What are the isothermal equations used in AKTS software to fit and project drug product stability data? What can I do to implement these in python in a performant manner?

Response: AKTS (Advanced Kinetics and Technology Solutions) software uses a combination of **isoconversional (model-free) kinetics** and **model-fitting reaction kinetic equations** coupled with the **Arrhenius law** to fit stability and degradation data, allowing projections under isothermal or climate-controlled storage conditions.

1. Core Kinetic Equations Used in AKTS

The Fundamental Rate Equation

The rate of degradation or conversion α (where $\alpha \in [0, 1]$ represents the extent of reaction or fraction decomposed) is expressed as:

$$\frac{d\alpha}{dt} = k(T) \cdot f(\alpha)$$

Where:

- $k(T)$ is the temperature-dependent rate constant given by the Arrhenius equation:

$$k(T) = A \exp\left(-\frac{E_a}{RT}\right)$$

- A : Pre-exponential factor (s^{-1} or concentration-dependent)
 - E_a : Activation energy (J/mol)
 - R : Universal gas constant (8.314 J/(mol · K))
 - T : Absolute temperature (K)
- $f(\alpha)$ is the reaction model function representing the degradation mechanism.

Common Reaction Models $f(\alpha)$ & Integrated Forms $g(\alpha)$

In isothermal conditions, integrating $\int_0^\alpha \frac{d\alpha}{f(\alpha)} = g(\alpha) = k(T) \cdot t$:

Model	Mechanism	$f(\alpha)$	$g(\alpha) = k(T) \cdot t$
F_0	Zero-Order	1	α
F_1	First-Order	$1 - \alpha$	$-\ln(1 - \alpha)$
F_n	n^{th} -Order	$(1 - \alpha)^n$	$\frac{1 - (1 - \alpha)^{1-n}}{1-n}$
A_2	Avrami-Erofeev ($n = 2$)	$2(1 - \alpha)[-\ln(1 - \alpha)]^{1/2}$	$[-\ln(1 - \alpha)]^{1/2}$
R_n	Contracting Sphere/Cylinder	$n(1 - \alpha)^{1-1/n}$	$1 - (1 - \alpha)^{1/n}$
B_{na}	Prout-Tompkins (Autocatalytic)	$\alpha^a(1 - \alpha)^n$	Numerical integration

Differential Isoconversional Method (Friedman Method)

AKTS relies heavily on the **Friedman model-free differential method** to avoid assuming a single reaction order across varying temperatures:

$$\ln\left(\frac{d\alpha}{dt}\right)_{\alpha,i} = \ln[A_\alpha f(\alpha)] - \frac{E_\alpha}{RT_{i,\alpha}}$$

For each degree of conversion α , E_α and $A_\alpha f(\alpha)$ are determined independently from isothermal/non-isothermal runs at temperature T_i .

Isothermal Shelf-Life Projection

For a target storage temperature T_{target} , the time required to reach a specific conversion level α (e.g., 5% degradation or $\alpha = 0.05$) is calculated as:

$$t(\alpha, T_{\text{target}}) = \frac{g(\alpha)}{A \exp\left(-\frac{E_a}{RT_{\text{target}}}\right)} = \int_0^\alpha \frac{d\alpha}{A_\alpha f(\alpha) \exp\left(-\frac{E_\alpha}{RT_{\text{target}}}\right)}$$

2. High-Performance Python Implementation

To build a fast, performant implementation in Python:

1. **Vectorize non-linear calculations** with NumPy.
2. **Accelerate ODE integration and fitting routines** with SciPy (`scipy.optimize.curve_fit / least_squares` or `scipy.integrate.solve_ivp`).
3. **Compile bottlenecks** using Numba JIT compilation to obtain C-like loop performance.

Python Example: Vectorized & JIT-Accelerated Isothermal Fit

```
import numpy as np
import scipy.optimize as opt
from numba import jit

# 1. High-Performance Arrhenius Rate Constant Calculation (JIT Compiled)
@jit(nopython=True, fastmath=True)
def arrhenius_k(A, Ea, T):
    """
    A : Pre-exponential factor (1/s)
    Ea : Activation Energy (J/mol)
    T : Temperature in Kelvin (K)
    """
    R = 8.31446261815324
    return A * np.exp(-Ea / (R * T))

# 2. Vectorized Integrated Conversion g(alpha) Functions
@jit(nopython=True, fastmath=True)
def g_alpha(alpha, model_type='F1', n=1.0):
    """Computes g(alpha) for various kinetic models."""
    if model_type == 'F0':
        return alpha
    elif model_type == 'F1':
        return -np.log(1.0 - alpha)
    elif model_type == 'Fn':
        return (1.0 - (1.0 - alpha)**(1.0 - n)) / (1.0 - n)
    elif model_type == 'A2':
        return np.sqrt(-np.log(1.0 - alpha))
    else:
        raise ValueError("Unsupported model type")

# 3. Predict Shelf-Life Time t(alpha)
@jit(nopython=True, fastmath=True)
def predict_isothermal_time(alpha_target, A, Ea, T_target, model_type='F1', n=1.0):
    """
    Calculates time to reach target conversion level alpha at storage temperature T_target.
    """
    k = arrhenius_k(A, Ea, T_target)
    g_val = g_alpha(alpha_target, model_type=model_type, n=n)
    return g_val / k

# 4. Global Nonlinear Multi-Temperature Fitting
def fit_stability_data(time_data, temp_data, alpha_data, model_type='F1'):
    """
    Fits multi-temperature stability datasets simultaneously for global parameters (A, Ea).
    time_data : 1D array of times
    temp_data : 1D array of temperatures in Kelvin
    alpha_data: 1D array of measured degradation fraction
    """
    def residual(params):
        log_A, Ea = params
        A = np.exp(log_A)
        R = 8.31446261815324

        # Calculate theoretical time for observed alpha values
        k_vals = A * np.exp(-Ea / (R * temp_data))
        g_vals = np.array([g_alpha(a, model_type=model_type) for a in alpha_data])
        predicted_time = g_vals / k_vals
```

```

    return predicted_time - time_data

# Initial guess: log(A) ~ 25, Ea ~ 80,000 J/mol
p0 = [25.0, 80000.0]
res = opt.least_squares(residual, p0, method='lm')

log_A_opt, Ea_opt = res.x
return np.exp(log_A_opt), Ea_opt

# --- Example Run ---
if __name__ == "__main__":
    # Synthetic dataset: 3 temperatures (300K, 310K, 320K)
    T_exp = np.array([300, 300, 310, 310, 320, 320], dtype=np.float64)
    alpha_exp = np.array([0.02, 0.05, 0.03, 0.08, 0.05, 0.12], dtype=np.float64)
    time_exp = np.array([120, 310, 45, 125, 18, 46], dtype=np.float64) # hours

    # Fit parameters globally
    A_fit, Ea_fit = fit_stability_data(time_exp, T_exp, alpha_exp, model_type='F1')
    print(f"Fitted A : {A_fit:.3e} s^-1")
    print(f"Fitted Ea: {Ea_fit/1000:.2f} kJ/mol")

    # Project Shelf-Life (5% degradation at 298.15 K / 25°C)
    t_shelf_25C = predict_isothermal_time(0.05, A_fit, Ea_fit, 298.15, model_type='F1')
    print(f"Projected Shelf-Life at 25°C (5% degradation): {t_shelf_25C:.1f} hours")

```

3. Key Optimization Strategies for Production

- **Global Model Optimization:** Use `scipy.optimize.least_squares` with the Levenberg-Marquardt (lm) method for fast, robust convergence when fitting multiple isothermal curves simultaneously.
- **Isoconversional Friedman Splines:** For model-free approaches without assumed $f(\alpha)$, fit univariate smoothing splines (`scipy.interpolate.UnivariateSpline`) to conversion curves to extract continuous $d\alpha/dt$ derivatives smoothly before performing linear regressions for E_α .
- **Avoid Python Loops:** Vectorize temperature grids using NumPy broadcasting to evaluate climate temperature profiles (e.g., daily fluctuations) across thousands of time steps in milliseconds.

For a deeper dive into the kinetic evaluation procedures used in AKTS, you can watch this [AKTS Thermokinetics Software Kinetic Analysis Tutorial](#). This video demonstrates the workflow for baseline construction, parameter evaluation, and reaction prediction.

Sources

1. [AKTS: Home Page - Advanced Kinetics and Technology Solutions](#)
2. [AKTS - THERMOKINETICS Software \(TK\) - General Overview](#)

User prompt: There is documentation regarding it get parameters m and n . Where are these in your response?

Response: In AKTS, parameters m and n come from the **Sesták–Berggren $SB(m, n)$ model** (also known as the generalized reaction order / autocatalytic model), which is one of the primary reaction mechanisms built directly into AKTS for fitting complex, multi-stage degradation processes.

1. Where m and n Fit into the Rate Equation

The standard rate equation is:

$$\frac{d\alpha}{dt} = A \exp\left(-\frac{E_a}{RT}\right) \cdot f(\alpha)$$

In the **Sesták–Berggren model $SB(m, n)$** , the reaction model function $f(\alpha)$ is explicitly defined as:

$$f(\alpha) = \alpha^m (1 - \alpha)^n$$

Where:

- n : Represents the standard **reaction order** (similar to First-Order F_1 or n -th order kinetics).
- m : Represents the **autocatalytic exponent**. When $m > 0$, the reaction rate initially starts low, accelerates as degradation product forms (α^m), hits a peak rate, and then decreases as reactants deplete ($(1 - \alpha)^n$).

2. Combined Sesták–Berggren Variant: $SB(m, n, p)$

AKTS also frequently uses an expanded variant with an additional exponent p :

$$f(\alpha) = \alpha^m (1 - \alpha)^n [-\ln(1 - \alpha)]^p$$

Exponent	Physical Meaning / Kinetic Behavior
n	Rate dependency on remaining concentration/reactant
m	Autocatalytic acceleration caused by formed degradation product
p	Nucleation / diffusion growth exponent

When $m = 0$ and $p = 0$, $f(\alpha) = (1 - \alpha)^n$, which collapses back to standard n^{th} -order kinetics (F_n).

3. High-Performance Python Implementation with m and n

Because $f(\alpha) = \alpha^m (1 - \alpha)^n$ does not generally have a closed-form analytical integral $g(\alpha)$, we evaluate $\frac{d\alpha}{dt}$ directly and use numerical integration (or `scipy.integrate.solve_ivp`) to project degradation over time.

Below is an optimized implementation using Numba and SciPy to solve and fit $SB(m, n)$ kinetics efficiently:

```
import numpy as np
import scipy.optimize as opt
from scipy.integrate import solve_ivp
from numba import jit

# 1. JIT-Accelerated Sestak-Berggren Reaction Rate
@jit(nopython=True, fastmath=True)
def sb_f_alpha(alpha, m, n):
    """
    Sestak-Berggren f(alpha) = alpha^m * (1 - alpha)^n
    Clamped to prevent negative or zero-division issues at boundaries.
    """
    alpha_clamped = max(1e-7, min(alpha, 1.0 - 1e-7))
    return (alpha_clamped ** m) * ((1.0 - alpha_clamped) ** n)

# 2. ODE Derivative function for solve_ivp
def dalpha_dt(t, alpha, A, Ea, m, n, T):
    R = 8.31446261815324
    k = A * np.exp(-Ea / (R * T))
    return k * sb_f_alpha(alpha[0], m, n)

# 3. Fast Isothermal Predictor for SB(m, n)
def predict_sb_degradation(t_span, alpha0, A, Ea, m, n, T):
    """
    Integrates d(alpha)/dt over time t_span at temperature T (Kelvin).
    """
    sol = solve_ivp(
        fun=dalpha_dt,
        t_span=(t_span[0], t_span[-1]),
        y0=[alpha0],
        t_eval=t_span,
        args=(A, Ea, m, n, T),
        method='RK45',
        rtol=1e-6,
        atol=1e-8
    )
    return sol.y[0]

# 4. Fitting Parameters (A, Ea, m, n) to Multi-Temperature Experimental Data
def fit_sb_parameters(time_grid, temp_grid, alpha_observed):
    """
    Fits log(A), Ea, m, and n simultaneously using non-linear least squares.
    """
    def residual(params):
        log_A, Ea, m, n = params
        A = np.exp(log_A)
```

```
alpha_pred = []

# Integrate each temperature experiment
for i, T in enumerate(temp_grid):
    t_eval = time_grid[i]
    y0 = alpha_observed[i][0]
    y_fit = predict_sb_degradation(t_eval, y0, A, Ea, m, n, T)
    alpha_pred.extend(y_fit - alpha_observed[i])

return np.array(alpha_pred)

# Initial Guesses: [log(A), Ea (J/mol), m, n]
p0 = [25.0, 85000.0, 0.5, 1.0]
bounds = ([0.0, 10000.0, 0.0, 0.0], [100.0, 300000.0, 5.0, 5.0])

res = opt.least_squares(residual, p0, bounds=bounds, method='trf')
log_A, Ea, m, n = res.x
return np.exp(log_A), Ea, m, n

# --- Example Run ---
if __name__ == "__main__":
    # Simulate an autocatalytic degradation process at 25°C (298.15 K)
    A_true, Ea_true = 1e11, 75000.0 # s^-1, J/mol
    m_true, n_true = 0.4, 1.2

    t_span = np.linspace(0, 365 * 24 * 3600, 100) # 1 year in seconds
    alpha_proj = predict_sb_degradation(t_span, 0.001, A_true, Ea_true, m_true, n_true, 298.15)

    print(f"Projected degradation after 1 year at 25°C: {alpha_proj[-1]*100:.2f}%")
```

User prompt: What are some analytical models (non-ode) that can be used with global fitting?

Response: To perform **global fitting** on drug stability datasets across multiple temperatures (and humidities) without integrating differential equations numerically at runtime, you can use **closed-form analytical solutions**.

Analytical models express degradation directly as a function of time: $\alpha(t) = g^{-1}(k \cdot t)$, where the rate constant $k(T, RH)$ is governed by the Arrhenius or ASAP (Accelerated Stability Assessment Program) equations.

The primary non-ODE analytical models suitable for global fitting are listed below.

1. Standard Integrated Kinetic Models

For standard reaction models where $g(\alpha) = \int_0^\alpha \frac{d\alpha}{f(\alpha)}$ can be inverted analytically, $\alpha(t)$ is expressed explicitly as a function of time t .

Model	Integrated Form $g(\alpha)$	Analytical Explicit Form $\alpha(t)$	Physical Mechanism
Zero-Order (F_0)	α	$\alpha(t) = k \cdot t$	Constant release / constant degradation rate
First-Order (F_1)	$-\ln(1 - \alpha)$	$\alpha(t) = 1 - e^{-k \cdot t}$	Solution-state decomposition, single reactant
Second-Order (F_2)	$\frac{\alpha}{1-\alpha}$	$\alpha(t) = \frac{k \cdot t}{1 + k \cdot t}$	Bimolecular interaction / dimer formation
n^{th} -Order ($F_n, n \neq 1$)	$\frac{1-(1-\alpha)^{1-n}}{1-n}$	$\alpha(t) = 1 - [1 - (1 - n)k \cdot t]^{\frac{1}{1-n}}$	General concentration-dependent reaction order

2. Solid-State Nucleation and Growth Models

Solid-state kinetics (crystallization, hydrate loss, polymorphic transitions) often follow closed-form analytical functions derived from geometric or nucleation assumptions:

Avrami–Erofeev Models (A_n)

Describes reactions driven by random nucleation and multidimensional growth.

$$\alpha(t) = 1 - \exp [-(k \cdot t)^n]$$

- $n = 2$ (A_2): 2D growth / disc-shaped nuclei.

- $n = 3$ (A_3): 3D growth / spherical nuclei.
- $n = 4$ (A_4): 3D growth with accelerating nucleation rate.

Contracting Geometry Models (R_n)

Describes reactions initiating on the surface and moving inward toward the center of the particle.

- **Contracting Area / Cylinder (R_2):** $\alpha(t) = 1 - (1 - k \cdot t)^2$
- **Contracting Volume / Sphere (R_3):** $\alpha(t) = 1 - (1 - k \cdot t)^3$

3. Empirical & Moisture-Sensitive Models

Modified ASAP (Waterman) Model

The Accelerated Stability Assessment Program (ASAP) equation incorporates both temperature and relative humidity (RH) into an analytical expression:

$$k(T, RH) = A \cdot \exp\left(-\frac{E_a}{RT} + B \cdot RH\right)$$

Where B is the **humidity sensitivity factor** ($\%RH^{-1}$). Substituting $k(T, RH)$ into a First-Order analytical expression gives the explicit model:

$$\alpha(t, T, RH) = 1 - \exp\left[-A \cdot t \cdot \exp\left(-\frac{E_a}{RT} + B \cdot RH\right)\right]$$

Extended Prout–Tompkins (Simplified Autocatalytic)

While general autocatalytic kinetics require ODEs, a simplified analytical form for sigmoidal degradation curves exists:

$$\alpha(t) = \frac{1}{1 + \exp[-k(t - t_{1/2})]}$$

4. High-Performance Python Implementation (Global Fitting)

By combining analytical equations, we can fit parameters (A , E_a , B , n) **globally** across an entire dataset of multiple temperatures and humidities in a single step using vectorized calculations without numerical ODE solver overhead.

```
import numpy as np
import scipy.optimize as opt

# 1. Analytical Degradation Functions
def alpha_analytical(t, T, RH, log_A, Ea, B, model='F1', n=1.0):
    """
    Computes alpha(t) analytically using the Waterman-Arrhenius rate constant:
    k(T, RH) = exp(log_A) * exp(-Ea/(R*T) + B*RH)
    """
    R = 8.31446261815324
    A = np.exp(log_A)

    # Vectorized rate constant calculation
    k = A * np.exp(-Ea / (R * T) + B * RH)
    kt = k * t

    if model == 'F0':
        return np.minimum(1.0, kt)
    elif model == 'F1':
        return 1.0 - np.exp(-kt)
    elif model == 'F2':
        return kt / (1.0 + kt)
    elif model == 'Fn':
        return 1.0 - np.maximum(0.0, 1.0 - (1.0 - n) * kt)**(1.0 / (1.0 - n))
    elif model == 'Avrami2': # A2 model
        return 1.0 - np.exp(-(kt)**2)
    elif model == 'R3': # Contracting sphere
        return 1.0 - np.maximum(0.0, 1.0 - kt)**3
    else:
        raise ValueError("Unknown model specified")
```

```
# 2. Global Fitting Function across Multi-T/RH Experiments
def fit_global_stability(time_data, temp_data, rh_data, alpha_data, model='F1'):
    """
    Global non-linear least squares optimization without ODE integration.
    """
    def residuals(params):
        if model == 'Fn':
            log_A, Ea, B, n = params
        else:
            log_A, Ea, B = params
            n = 1.0

        pred_alpha = alpha_analytical(time_data, temp_data, rh_data, log_A, Ea, B, model=model, n=n)
        return pred_alpha - alpha_data

    # Initial guesses: [log(A), Ea (J/mol), B (%RH^-1)]
    if model == 'Fn':
        p0 = [25.0, 80000.0, 0.03, 1.5]
        bounds = ([0, 10000, 0, 0.1], [100, 300000, 0.2, 5.0])
    else:
        p0 = [25.0, 80000.0, 0.03]
        bounds = ([0, 10000, 0], [100, 300000, 0.2])

    res = opt.least_squares(residuals, p0, bounds=bounds, method='trf')

    fit_results = {
        'A': np.exp(res.x[0]),
        'Ea_kJ': res.x[1] / 1000.0,
        'B': res.x[2]
    }
    if model == 'Fn':
        fit_results['n'] = res.x[3]

    return fit_results

# --- Example Run ---
if __name__ == "__main__":
    # Multi-condition dataset: T (K), RH (fraction 0-100), Time (days), Alpha (fraction)
    T_exp = np.array([313.15, 313.15, 323.15, 323.15, 333.15, 333.15]) # 40C, 50C, 60C
    RH_exp = np.array([75.0, 75.0, 50.0, 50.0, 11.0, 11.0]) # % RH
    t_exp = np.array([14.0, 30.0, 14.0, 30.0, 14.0, 30.0]) # days

    # Synthetic degradation observations
    alpha_obs = np.array([0.021, 0.043, 0.035, 0.072, 0.051, 0.105])

    # Perform global analytical fit
    results = fit_global_stability(t_exp, T_exp, RH_exp, alpha_obs, model='F1')

    print("--- Global Analytical Fit Results ---")
    print(f"Pre-exponential A : {results['A']:.3e} day^-1")
    print(f"Activation Energy : {results['Ea_kJ']:.2f} kJ/mol")
    print(f"RH Coefficient B : {results['B']:.4f} %RH^-1")
```

5. Benefits of Analytical Models over ODEs for Global Fitting

- 1. Extreme Execution Speed:** Evaluation takes microsecond durations because functions are purely array operations natively parallelized by NumPy/BLAS.
- 2. Exact Gradient Computation:** Jacobian matrices can be computed analytically, ensuring faster and more stable convergence with solvers like Levenberg-Marquardt (lm) or Trust Region Reflective (trf).
- 3. No Numerical Stiff/Step-Size Errors:** Eliminates solver tolerance issues (rtol/atol) that occasionally plague complex ODE integrators during parameter space exploration.

User prompt: Can optuna or similar be used to accelerate ode calcs?

Response: No, Optuna itself cannot accelerate ODE calculations. Optuna is a hyperparameter optimization and parameter search framework—it decides *which* parameters (A , E_a , m , n) to evaluate next, but it relies on your objective function (which calls the ODE solver) to compute each step.

However, you can achieve **10x to 100x speedups** for ODE calculations in Python by using the following techniques and tools alongside Optuna:

1. How to Actually Accelerate Python ODE Calculations

A. Use Compiled / JIT Solvers (difffrax + JAX or torchdiffeq)

Standard `scipy.integrate.solve_ivp` runs in pure Python and C wrappers, which carries significant per-call overhead when called thousands of times inside optimization loops.

Instead, use `difffrax` (built on Google JAX) or `torchdiffeq` (built on PyTorch):

- **JIT Compilation (XLA):** Compiles the entire differential equation integrator into raw machine code (or GPU instructions).
- **Automatic Differentiation (Autodiff):** Allows exact parameter gradients $\nabla_{\theta}\alpha(t)$ via adjoint state methods, avoiding expensive finite-difference approximations.

B. C/C++ JIT Compilation via Numba or Cython

If staying in the NumPy ecosystem, write the ODE RHS function using Numba JIT and pass it to a high-speed compiled solver like C++ `Boost.Numeric.Odeint` or SciPy's `LowLevelCallable` wrappers.

2. Where Optuna Does Help: Smarter, Fewer Evaluations

While Optuna doesn't speed up individual ODE runs, it dramatically reduces the **total number of ODE solves needed** compared to traditional grid search or standard non-linear least squares (`scipy.optimize.least_squares`):

1. **Tree-structured Parzen Estimator (TPE):** Uses Bayesian optimization to navigate complex, highly correlated parameter spaces (like Arrhenius $\ln A$ vs E_a) far faster than traditional global solvers.
2. **Pruning (Hyperband / Median Pruner):** Stops solving an ODE early if the error in early time steps is already worse than the best prior runs.
3. **Parallel Execution:** Native multi-processing (`n_jobs=-1`) allows running parallel ODE solves across multiple CPU cores or GPUs simultaneously.

3. High-Performance Example: Optuna + Difffrax (JAX JIT ODEs)

Here is a complete, performant pattern using Optuna for intelligent parameter search and Difffrax for JIT-compiled ODE integration:

```
import optuna
import jax
import jax.numpy as jnp
from difffrax import difffrax, ODETerm, Dopri5, SaveAt

# Optuna logging verbosity
optuna.logging.set_verbosity(optuna.logging.WARNING)

# 1. Define JAX JIT-compiled Sestak-Berggren ODE system
def sb_ode(t, y, args):
    log_A, Ea, m, n, T = args
    R = 8.31446261815324

    A = jnp.exp(log_A)
    k = A * jnp.exp(-Ea / (R * T))

    alpha = jnp.clip(y[0], 1e-7, 1.0 - 1e-7)
    dalpha = k * (alpha**m) * ((1.0 - alpha)**n)
    return jnp.array([dalpha])

# 2. Fully JIT-compiled ODE trajectory solver across multiple temperatures
@jax.jit
def solve_multi_temp(log_A, Ea, m, n, temps, t_eval):
    def solve_single_T(T):
        term = ODETerm(sb_ode)
        solver = Dopri5()
        # Solve ODE using Difffrax
        sol = difffrax.diffeqsolve(
            term, solver, t0=t_eval[0], t1=t_eval[-1], dt0=0.1,
            y0=jnp.array([0.001]),
            args=(log_A, Ea, m, n, T),
            saveat=SaveAt(ts=t_eval),
            max_steps=1000
        )
    return sol.ys[:, 0]

# Vectorize across all temperature conditions automatically
```

```

    return jax.vmap(solve_single_T)(temps)

# 3. Optuna Objective Function
def objective(trial, temps, t_eval, alpha_experimental):
    # Parameter space search
    log_A = trial.suggest_float("log_A", 10.0, 40.0)
    Ea = trial.suggest_float("Ea", 40000.0, 150000.0)
    m = trial.suggest_float("m", 0.0, 3.0)
    n = trial.suggest_float("n", 0.1, 3.0)

    # Fast JIT ODE evaluation across all T
    pred_alpha = solve_multi_temp(log_A, Ea, m, n, temps, t_eval)

    # Compute Mean Squared Error (MSE)
    mse = jnp.mean((pred_alpha - alpha_experimental) ** 2)
    return float(mse)

# --- Execution Pipeline ---
if __name__ == "__main__":
    # Multi-temperature setup (300K, 310K, 320K)
    temps = jnp.array([300.0, 310.0, 320.0])
    t_eval = jnp.linspace(0, 100, 50) # 50 timepoints

    # Generate synthetic target data for testing
    true_log_A, true_Ea, true_m, true_n = 25.0, 80000.0, 0.5, 1.2
    alpha_exp = solve_multi_temp(true_log_A, true_Ea, true_m, true_n, temps, t_eval)

    # Warmup JIT compilation step
    _ = solve_multi_temp(20.0, 70000.0, 0.5, 1.0, temps, t_eval)

    # Run Optuna parameter optimization
    study = optuna.create_study(direction="minimize")
    study.optimize(
        lambda trial: objective(trial, temps, t_eval, alpha_exp),
        n_trials=500,
        n_jobs=1 # JAX handles internal SIMD parallelization
    )

    print("--- Optuna Fit Optimization Complete ---")
    print("Best Params Found:")
    for k, v in study.best_params.items():
        print(f"  {k}: {v:.4f}")
```

4. Summary Recommendation

Tool Combination	Typical Speed per 1000 Solves	Best Use Case
SciPy (solve_ivp) + SciPy least_squares	~5.0 - 15.0 seconds	Quick baseline scripts
Numba JIT ODE + SciPy least_squares	~0.5 - 1.5 seconds	Mid-sized models without extra dependencies
Optuna + JAX/Diffrax (jax.jit + jax.vmap)	~0.01 - 0.05 seconds	High-throughput global kinetic fitting

User prompt: Now if I blindly fit all of these models to data, how do I select the best one?

Response: Selecting the best kinetic model from a multi-model fit requires balancing **goodness-of-fit**, **model complexity**, and **physical/chemical plausibility**. Fitting algorithms will always give a lower residual sum of squares (RSS) to models with more parameters (e.g., $SB(m, n)$ will almost always fit better than F_1), but overparameterized models lead to poor shelf-life projections.

Here is the standard workflow to evaluate and rank candidate models automatically.

1. Information Theoretic Criteria (Penalizing Complexity)

Never rely solely on R^2 or RSS. Instead, use **Akaike Information Criterion (AIC)** or **Bayesian Information Criterion (BIC)**. These metrics trade off fitting accuracy against parameter count (k), penalizing overparameterization.

Akaike Information Criterion with Correction (AIC_c)

Use AIC_c when your sample size N (total data points) is small relative to the number of parameters k ($N/k < 40$):

$$AIC_c = N \cdot \ln \left(\frac{RSS}{N} \right) + 2k + \frac{2k(k+1)}{N-k-1}$$

Where:

- N : Total number of data points across all temperatures/conditions.
- RSS : Residual Sum of Squares $\sum (\alpha_{\text{measured}} - \alpha_{\text{predicted}})^2$.
- k : Number of fitted parameters (e.g., F_1 has $k = 2$ [A, E_a]; $SB(m, n)$ has $k = 4$ [A, E_a, m, n]).

Selection Rule: The model with the **lowest** AIC_c is preferred.

Model Probability / Akaike Weights (w_i)

To quantify *how much better* the winning model is compared to the rest, calculate Akaike weights:

$$\Delta_i = AIC_{c,i} - AIC_{c,\min}$$
$$w_i = \frac{\exp \left(-\frac{1}{2} \Delta_i \right)}{\sum_j \exp \left(-\frac{1}{2} \Delta_j \right)}$$

w_i represents the probability that Model i is truly the best model among the candidate set.

2. Statistical Criteria Matrix

Combine the following quantitative metrics into a scoring matrix:

Metric	Target	Purpose
AIC_c / BIC	Minimum	Best balance between fitting accuracy and model parsimony.
Adjusted R^2 (R^2_{adj})	> 0.98	Ensures the fit accounts for variance while adjusting for degrees of freedom.
Root Mean Square Error (RMSE)	Minimum	Direct error magnitude in degradation units (α).
Durbin-Watson Statistic (DW)	≈ 2.0	Checks for autocorrelation in residuals (ensures errors are non-systematic).

3. Physical & Thermodynamic Sanity Checks

Even if a model wins on AIC_c , it **must be rejected** if it violates chemical principles:

1. **Activation Energy (E_a):** For typical pharmaceutical degradation mechanisms (hydrolysis, oxidation, thermolysis), E_a should generally fall between **50 and 120 kJ/mol**.
 - $E_a < 30$ kJ/mol: Unphysically low (indicates pure diffusion control or model misfitting).
 - $E_a > 180$ kJ/mol: Extremely high; often indicates overparameterization or baseline instability.
2. **Pre-Exponential Factor ($\ln A$):** Must be positive and realistically proportioned relative to E_a via the compensation effect.
3. **Exponents (m, n):** In $SB(m, n)$ or F_n models, kinetic exponents should generally be bounded within reasonable limits ($0 \leq m \leq 2$ and $0.1 \leq n \leq 3$).

4. Cross-Validation (Extrapolation Stability)

To ensure the model projects shelf-life reliably rather than just memorizing the high-temperature stress data:

- **Leave-One-Temperature-Out Cross-Validation (LOTOCV):** Fit the model on $N - 1$ temperature datasets (e.g., 40°C, 50°C, 60°C) and test its prediction accuracy on the omitted temperature dataset (e.g., 25°C or 30°C). Models that overfit stress data perform poorly when predicting lower-temperature behavior.

5. Python Implementation: Automatic Model Ranker

Here is a complete, automated model selection script in Python:

```

import numpy as np

def evaluate_model_fit(N, RSS, k, model_name):
    """
    Computes statistical evaluation metrics for a kinetic model fit.

    N          : int    - Total number of data points
    RSS         : float  - Residual sum of squares
    k           : int    - Number of estimated parameters
    model_name: str     - Identifier string
    """
    # Root Mean Square Error
    rmse = np.sqrt(RSS / N)

    # AIC and AICc (Corrected AIC for small sample sizes)
    aic = N * np.log(RSS / N) + 2 * k
    if (N - k - 1) > 0:
        aicc = aic + (2 * k * (k + 1)) / (N - k - 1)
    else:
        aicc = np.inf

    # BIC (Bayesian Information Criterion)
    bic = N * np.log(RSS / N) + k * np.log(N)

    return {
        'model': model_name,
        'k': k,
        'RSS': RSS,
        'RMSE': rmse,
        'AICc': aicc,
        'BIC': bic
    }

def rank_candidate_models(results_list):
    """
    Ranks models by AICc and computes Akaike weights (probabilities).
    """
    # Sort by AICc ascending
    sorted_results = sorted(results_list, key=lambda x: x['AICc'])
    min_aicc = sorted_results[0]['AICc']

    # Compute delta AICc and unnormalized weights
    for res in sorted_results:
        res['delta_AICc'] = res['AICc'] - min_aicc
        res['rel_likelihood'] = np.exp(-0.5 * res['delta_AICc'])

    sum_likelihood = sum(r['rel_likelihood'] for r in sorted_results)

    # Compute Akaike Weights (Model Probability)
    for res in sorted_results:
        res['akaike_weight'] = res['rel_likelihood'] / sum_likelihood

    return sorted_results

# --- Example Evaluation ---
if __name__ == "__main__":
    N_obs = 36 # Total observations across all temperatures

    # Simulated residual outcomes from fitting candidate models
    fits = [
        evaluate_model_fit(N_obs, RSS=0.00120, k=2, model_name="Zero-Order (F0)"),
        evaluate_model_fit(N_obs, RSS=0.00045, k=2, model_name="First-Order (F1)"),
        evaluate_model_fit(N_obs, RSS=0.00042, k=3, model_name="n-th Order (Fn)"),
        evaluate_model_fit(N_obs, RSS=0.00038, k=4, model_name="Sestak-Berggren SB(m,n)"),
        evaluate_model_fit(N_obs, RSS=0.00048, k=2, model_name="Avrami-Erofeev (A2)")
    ]

    ranked = rank_candidate_models(fits)

    print(f"{'Rank':<5} | {'Model':<22} | {'k':<3} | {'RMSE':<8} | {'AICc':<8} | {'Prob (Weight)':<12}")
    print("-" * 70)
    for i, r in enumerate(ranked, 1):
        print(f"{i:<5} | {r['model']:<22} | {r['k']:<3} | {r['RMSE']:.5f} | {r['AICc']:.2f} | {r['akaike_weight']*100:.")

```

Summary Selection Rule of Thumb

1. Calculate AIC_c for all models.

2. Filter out models with **unrealistic** E_a (< 30 or > 180 kJ/mol).

3. Pick the model with the **lowest** AIC_c .

4. If a simpler model (e.g., F_1) has a $\Delta AIC_c < 2.0$ compared to a complex model (e.g., $SB(m, n)$), **prefer the simpler model** to avoid overparameterization in regulatory submissions.

User prompt: What are the regulatory constraints and ICH Q1A/Q1E guidelines regarding statistical extrapolation of shelf-life data?

Response: The International Council for Harmonisation (ICH) guidelines—specifically **ICH Q1A(R2)** (*Stability Testing of New Drug Substances and Products*) and **ICH Q1E** (*Evaluation of Stability Data*)—regulate how stability data can be statistically evaluated and extrapolated to establish retest periods or shelf-lives.

Regulatory agencies (FDA, EMA, PMDA) enforce strict limits on how far beyond real-time data you can project, as well as the statistical rigor required to justify those extrapolations.

1. Core Principles: ICH Q1A(R2) & ICH Q1E

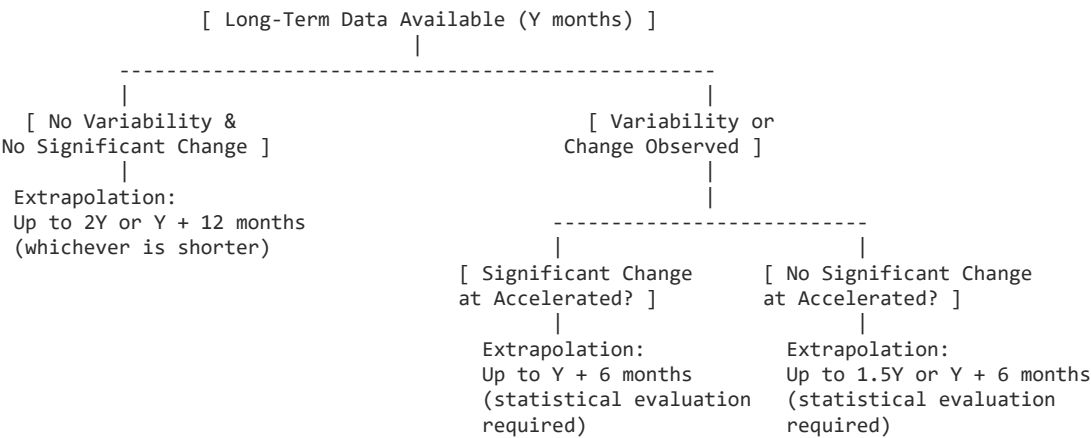
Real-Time vs. Accelerated Data

- Accelerated Data (e.g., 40°C / 75% RH):** Primary purpose is to identify degradation products, evaluate stress behavior, and support short-term excursions (e.g., shipping). Accelerated data **cannot** be used as the sole basis to extrapolate a long-term shelf-life without supporting long-term data.
- Long-Term Data (e.g., 25°C / 60% RH or 30°C / 65% RH):** Long-term data drives the proposed shelf-life. Extrapolation is defined strictly as projecting a retest period or shelf-life *beyond* the time period covered by real-time long-term data.

2. Maximum Allowable Extrapolation Limits (ICH Q1E Decision Tree)

ICH Q1E specifies maximum extrapolation limits based on whether the data shows variability, significant change under stress, or requires statistical evaluation.

Let Y = duration of available long-term data (e.g., 12 months).



Summary of Max Shelf-Life Limits

Condition / Data Outcome	Maximum Proposed Shelf-Life	Example ($Y = 12$ months Long-Term Data)
No change at accelerated; negligible variability at long-term	$\min(2Y, Y + 12 \text{ months})$	24 months ($\min(24, 24)$)
No significant change at accelerated; long-term data variable	$\min(1.5Y, Y + 6 \text{ months})$	18 months ($\min(18, 18)$)
Significant change at accelerated (e.g., 5% assay loss, toxicity limit breach)	$\min(Y + 6 \text{ months}, 1.5Y)$	18 months (requires statistical statistical proof)

Condition / Data Outcome	Maximum Proposed Shelf-Life	Example ($Y = 12$ months Long-Term Data)
Significant change at accelerated AND failure at intermediate (30°C/65% RH)	No extrapolation allowed (Shelf-life = Y)	12 months

Key Rule: Extrapolation beyond $Y + 12$ months is **never allowed** for primary registration submissions, regardless of how perfect your Arrhenius or kinetic model fit is.

3. Statistical Requirements for Extrapolation

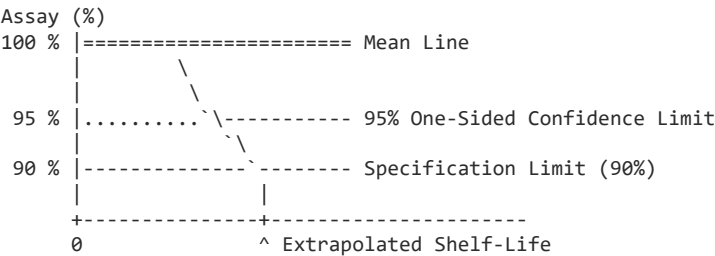
When statistical evaluation is required to justify an extrapolation, ICH Q1E specifies the following methodologies:

A. One-Sided 95% Confidence Bounds

For attributes that decrease over time (e.g., active ingredient Assay), or increase over time (e.g., Degradation Products/Impurities):

- You must construct a **one-sided 95% confidence limit** around the mean degradation curve.
- The extrapolated shelf-life is the timepoint where the **95% confidence bound crosses the acceptance criterion / specification limit**, *not* where the mean regression line crosses it.

Shelf-Life = Time t where $\hat{y}(t) \pm t_{0.05,\nu} \cdot SE(\hat{y}(t)) = \text{Specification Limit}$



B. Pooling Data Across Batches

ICH Q1E requires testing at least **3 primary batches**. To pool data across batches to create a single shelf-life estimate:

- Perform an **Analysis of Covariance (ANCOVA)** on batch slopes and intercepts.
- Test for equality of slopes and intercepts at a significance level of $\alpha = 0.25$ (a deliberately high threshold to prevent inappropriate pooling):
 - If $p > 0.25$: Slopes and intercepts can be pooled into a single combined regression model.
 - If $p \leq 0.25$: Batches cannot be pooled. The worst-performing batch dictates the shelf-life for the entire product.

4. How AKTS / Advanced Kinetic Models Fit into Regulatory Submissions

Advanced kinetic tools (like AKTS, ASAPprime, or custom Python models based on Arrhenius/Sesták–Berggren equations) are powerful, but regulators view them through a specific lens:

- Supporting/Excursion Role vs. Primary Shelf-Life:**
 - Advanced kinetic modeling is widely accepted for **designing stability protocols, justifying temperature excursion tolerances during shipping, and risk assessment/formulation screening**.
 - However, for **formal shelf-life assignment** in a Marketing Authorization Application (MAA/NDA), empirical real-time data combined with ICH Q1E confidence interval limits **always supersedes** pure kinetic predictions.
- Validation of Isothermal/Non-Isothermal Kinetics:**
 - If using kinetic models to support shelf-life requests or shelf-life extensions, regulators require **model verification**: you must present actual long-term real-time data points overlaid against the kinetic model predictions to prove accuracy.

5. Summary Matrix for Regulatory Compliance

Aspect	Regulatory Standard
Min. Long-Term Data at Submission	12 months (for standard process); 6 months (for accelerated pathways in some jurisdictions).

Aspect	Regulatory Standard
Primary Metric for Limit	One-sided 95% Confidence Bound crossing the spec boundary.
Max Extrapolation Ceiling	Absolute cap at $Y + 12$ months (or $2Y$, whichever is lower).
Batch Pooling Criterion	ANCOVA $p > 0.25$ for batch slope/intercept homogeneity.
Role of Kinetic Software (AKTS/ASAP)	Used as supportive evidence, excursion justification, and risk assessment—not a replacement for real-time ICH stability protocols.

User prompt: Show me how to code the ICH Q1E 95% one-sided confidence interval shelf-life intersection method in Python.

Response: Here is a complete, production-ready implementation of the **ICH Q1E one-sided 95% confidence interval intersection method** using NumPy, SciPy, and Matplotlib.

This code computes the linear regression of stability data (e.g., active drug assay over time), constructs the **one-sided 95% lower confidence bound** (for decreasing quality attributes like assay), and identifies the exact point where that bound intersects the specification limit.

Python Code: ICH Q1E 95% One-Sided Confidence Limit Shelf-Life Estimator

```
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt

def calculate_ich_shelflife(time_months, assay_percent, spec_limit=90.0, alpha=0.05, attribute_type='decrease'):
    """
    Calculates the ICH Q1E shelf-life based on the 95% one-sided confidence interval intersection.

    Parameters:
        time_months      : array-like - Measured time points (in months)
        assay_percent     : array-like - Measured stability values (e.g., Assay % or Impurity %)
        spec_limit        : float      - Acceptance specification limit (e.g., 90.0% for Assay, 1.0% for Impurity)
        alpha             : float      - Significance level (0.05 for one-sided 95% CI)
        attribute_type    : str        - 'decrease' (e.g., Assay) or 'increase' (e.g., Degradation/Impurity)

    Returns:
        dict: Fitted parameters, exact shelf-life, and evaluation grid for plotting.
    """
    x = np.asarray(time_months, dtype=np.float64)
    y = np.asarray(assay_percent, dtype=np.float64)
    n = len(x)

    if n < 3:
        raise ValueError("At least 3 data points are required for linear stability regression.")

    # 1. Ordinary Least Squares (OLS) Linear Regression
    x_mean = np.mean(x)
    y_mean = np.mean(y)

    Sxx = np.sum((x - x_mean)**2)
    Sxy = np.sum((x - x_mean) * (y - y_mean))

    slope = Sxy / Sxx
    intercept = y_mean - slope * x_mean

    # Residuals & Standard Error of the Estimate (Sy.x)
    y_pred = intercept + slope * x
    residuals = y - y_pred
    dof = n - 2

    residual_variance = np.sum(residuals**2) / dof
    se_estimate = np.sqrt(residual_variance)

    # 2. Critical t-value for One-Sided (1 - alpha) Confidence Interval
    t_crit = stats.t.ppf(1.0 - alpha, dof)

    # 3. Dense grid for evaluation & finding the intersection
    max_time_eval = max(np.max(x) * 2.5, 60.0) # Evaluate up to 60 months or 2.5x max data
    x_grid = np.linspace(0.0, max_time_eval, 2000)
    y_grid_mean = intercept + slope * x_grid
```

```

# Standard Error of Prediction Mean at each x_grid point
se_grid = se_estimate * np.sqrt((1.0 / n) + ((x_grid - x_mean)**2 / Sxx))

# 4. Compute One-Sided Confidence Limits
if attribute_type == 'decrease':
    # Lower 95% One-Sided Confidence Limit (For Assay / Potency)
    ci_bound = y_grid_mean - t_crit * se_grid
    # Intersection occurs when lower bound drops below spec_limit
    intersect_idx = np.where(ci_bound <= spec_limit)[0]
elif attribute_type == 'increase':
    # Upper 95% One-Sided Confidence Limit (For Impurities / Degradants)
    ci_bound = y_grid_mean + t_crit * se_grid
    # Intersection occurs when upper bound rises above spec_limit
    intersect_idx = np.where(ci_bound >= spec_limit)[0]
else:
    raise ValueError("attribute_type must be either 'decrease' or 'increase'")

# Determine Shelf-Life Time Point
if len(intersect_idx) > 0:
    shelf_life = x_grid[intersect_idx[0]]
else:
    shelf_life = max_time_eval # Did not cross limit within evaluation range

return {
    'slope': slope,
    'intercept': intercept,
    'se_estimate': se_estimate,
    'shelf_life': shelf_life,
    'x_data': x,
    'y_data': y,
    'x_grid': x_grid,
    'y_mean_grid': y_grid_mean,
    'ci_bound_grid': ci_bound,
    'spec_limit': spec_limit,
    'attribute_type': attribute_type
}

def plot_ich_shelflife(results):
    """Plots the ICH Q1E linear regression with one-sided 95% CI and shelf-life intersection."""
    res = results

    plt.figure(figsize=(10, 6))

    # Data points
    plt.scatter(res['x_data'], res['y_data'], color='navy', zorder=5, label='Observed Long-Term Data')

    # Regression mean line
    plt.plot(res['x_grid'], res['y_mean_grid'], color='blue', linestyle='--', linewidth=1.5, label='Mean Regression Line')

    # 95% One-Sided Confidence Limit
    ci_label = 'Lower 95% One-Sided CI' if res['attribute_type'] == 'decrease' else 'Upper 95% One-Sided CI'
    plt.plot(res['x_grid'], res['ci_bound_grid'], color='red', linewidth=2.0, label=ci_label)

    # Specification Limit Line
    plt.axhline(y=res['spec_limit'], color='black', linestyle=':', label=f"Spec Limit ({res['spec_limit']}%)")

    # Intersection Marker
    if res['shelf_life'] < np.max(res['x_grid']):
        plt.axvline(x=res['shelf_life'], color='green', linestyle='-', linewidth=1.5,
                    label=f"ICH Shelf-Life ({res['shelf_life']:.1f} Months)")
        plt.scatter([res['shelf_life']], [res['spec_limit']], color='green', s=100, zorder=6)

    plt.title("ICH Q1E Stability Evaluation & Shelf-Life Intersection", fontsize=13, fontweight='bold')
    plt.xlabel("Time (Months)", fontsize=11)
    plt.ylabel("Assay (%)" if res['attribute_type'] == 'decrease' else "Impurity (%)", fontsize=11)
    plt.xlim(0, max(np.max(res['x_data']) + 6, res['shelf_life'] + 4))

    # Adjust Y-limits dynamically
    y_min = min(np.min(res['y_data']), res['spec_limit'] - 2.0)
    y_max = max(np.max(res['y_data']), 102.0)
    plt.ylim(y_min, y_max)

```

```

plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(loc='lower left', frameon=True)
plt.tight_layout()
plt.show()

# --- Demonstration Run ---
if __name__ == "__main__":
    # Simulated 12-month long-term assay stability dataset (3 primary batches pooled)
    time_data = np.array([0, 0, 0, 3, 3, 3, 6, 6, 6, 9, 9, 9, 12, 12, 12])
    assay_data = np.array([
        100.2, 99.8, 100.5,
        99.1, 98.8, 99.4,
        98.2, 97.9, 98.5,
        97.1, 96.8, 97.4,
        96.0, 95.7, 96.3
    ])

    spec_limit = 90.0 # 90.0% minimum assay limit

    # Run ICH Q1E Evaluation
    results = calculate_ich_shelflife(time_data, assay_data, spec_limit=spec_limit, attribute_type='decrease')

    print("--- ICH Q1E Linear Regression Results ---")
    print(f"Intercept (Month 0) : {results['intercept']:.3f} %")
    print(f"Slope (Degradation) : {results['slope']:.4f} %/month")
    print(f"Std Error (Sy.x) : {results['se_estimate']:.4f}")
    print(f"Calculated Shelf-Life: {results['shelf_life']:.2f} Months")

    # Check max ICH Q1E regulatory ceiling (12 months data -> max 12 + 12 = 24 months)
    max_regulatory_claim = min(12 * 2, 12 + 12)
    stat_shelf_life = min(results['shelf_life'], max_regulatory_claim)
    print(f"Regulatory Max Shelf-Life (ICH Q1E Cap): {stat_shelf_life:.1f} Months")

    # Render Plot
    plot_ich_shelflife(results)

```

How Key ICH Q1E Rules Are Handled in Code

1. **One-Sided Student's *t*-Distribution (stats.t.ppf):** ICH Q1E requires a one-sided confidence interval ($\alpha = 0.05$, corresponding to 95% confidence).
2. **Expanding Standard Error Grid (se_grid):** Standard error increases as you extrapolate away from the mean time point ($\bar{x}$):

$$SE(\hat{y}_0) = S_{y \cdot x} \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum (x_i - \bar{x})^2}$$

This curvature correctly causes the confidence interval to flare outward, giving a conservative, statistically valid shelf-life estimate.

3. **Regulatory Cap Comparison:** The statistical intersection is capped against the regulatory ceiling ($\min(2Y, Y + 12 \text{ months})$), where Y is the duration of available real-time long-term data.

User prompt: Show me how to code the ICH Q1E 95% one-sided confidence interval shelf-life intersection method in Python.

Response: Here is an enhanced, complete Python implementation of the **ICH Q1E 95% one-sided confidence interval intersection method** that handles **both decreasing attributes** (e.g., active drug Assay)